

Module 13: UPDATE Statement

The **UPDATE** statement is used to modify existing data in a table. It is one of the most important Data Manipulation Language (DML) commands because it allows databases to remain accurate and up to date.



Introduction

In real-world databases, data changes frequently.

Examples:

- Employee salary increases
- Customer address changes
- Product price updates
- Order status changes

Instead of deleting and re-inserting records, SQL provides the:

UPDATE

statement.



Learning Objectives

After completing this module, you will be able to:

- ✓ Update single records
 - ✓ Update multiple records
 - ✓ Use WHERE with UPDATE
 - ✓ Update using calculations
 - ✓ Update multiple columns
 - ✓ Understand UPDATE risks
 - ✓ Apply best practices for production databases
-



Table of Contents

1. What is UPDATE?
2. Why UPDATE is Important
3. UPDATE Syntax
4. Updating a Single Record
5. Updating Multiple Records
6. UPDATE with WHERE
7. Updating Multiple Columns
8. UPDATE Using Expressions
9. UPDATE with CASE
10. UPDATE from Another Table
11. Common Mistakes
12. Best Practices
13. Real-World Examples
14. Summary
15. Practice Questions

1 What is UPDATE?

The **UPDATE** statement modifies existing rows in a table.

Example

Before Update:

EmployeeID	EmployeeName	Salary
101	Kunal	45000

Query:

```
UPDATE Employees
SET Salary = 50000
WHERE EmployeeID = 101;
```

After Update:

EmployeeID	EmployeeName	Salary
101	Kunal	50000

Why UPDATE Matters

Used daily in:

- HR Systems
- Banking Applications
- E-Commerce Platforms
- Inventory Systems
- CRM Software

2 Why UPDATE is Important

Imagine:

Employee:

Kunal

gets promoted.

Old Salary:

45000

New Salary:

60000

Instead of deleting the record, simply update it.

Benefits

- ✓ Faster
 - ✓ Safer
 - ✓ Preserves record history
 - ✓ Maintains relationships
-

3 UPDATE Syntax

Basic syntax:

```
UPDATE TableName  
SET ColumnName = Value  
WHERE Condition;
```

Example

```
UPDATE Employees  
SET Salary = 55000  
WHERE EmployeeID = 101;
```

General Structure

```
UPDATE  
SET  
WHERE
```

4 Updating a Single Record

Example Table

EmployeeID	EmployeeName	Salary
101	Kunal	45000
102	Rahul	50000

Query

```
UPDATE Employees  
SET Salary = 55000  
WHERE EmployeeID = 101;
```

Result

EmployeeID	EmployeeName	Salary
101	Kunal	55000
102	Rahul	50000

Important

The WHERE clause identifies which row should be updated.

5 Updating Multiple Records

An UPDATE can modify many rows at once.

Example

```
UPDATE Employees
SET City = 'Delhi'
WHERE Department = 'IT';
```

Before:

Employee	Department	City
Kunal	IT	NULL
Rahul	IT	NULL
Aman	HR	NULL

After:

Employee	Department	City
Kunal	IT	Delhi
Rahul	IT	Delhi
Aman	HR	NULL

6 UPDATE with WHERE

The WHERE clause controls which records are updated.

Example

```
UPDATE Employees
SET Salary = 60000
WHERE EmployeeName = 'Kunal';
```

Only matching records are updated.

Without WHERE

```
UPDATE Employees
SET Salary = 60000;
```

Result:

Every employee salary becomes 60000.

⚠ One of the most dangerous SQL mistakes.

7 Updating Multiple Columns

Multiple columns can be updated simultaneously.

Syntax

```
UPDATE TableName
SET Column1 = Value1,
    Column2 = Value2
WHERE Condition;
```

Example

```
UPDATE Employees
SET Salary = 65000,
    City = 'Mumbai'
WHERE EmployeeID = 101;
```

Before

EmployeeID	Salary	City
101	50000	Delhi

After

EmployeeID	Salary	City
101	65000	Mumbai

8 UPDATE Using Expressions

SQL can perform calculations during updates.

Example

Increase salary by ₹5000.

```
UPDATE Employees
SET Salary = Salary + 5000;
```

Before

Salary
45000
50000

After

Salary
50000
55000

Percentage Increase

10% salary hike:

```
UPDATE Employees
SET Salary = Salary * 1.10;
```

Example

Increase product prices by 5%.

```
UPDATE Products
SET Price = Price * 1.05;
```

Business Uses

- Salary increments
 - Price revisions
 - Tax adjustments
 - Inventory updates
-



UPDATE with CASE

Update different rows differently.

Example

```
UPDATE Employees
SET Salary =
CASE
    WHEN Department = 'IT'
    THEN Salary + 10000

    WHEN Department = 'HR'
    THEN Salary + 5000
```



```
    ELSE Salary
END;
```

Result

```
IT employees get ₹10000 increase.
HR employees get ₹5000 increase.
```

Why CASE?

Useful for:

- Department-specific changes
- Bonus calculations
- Category updates

10 UPDATE from Another Table

Common in enterprise databases.

Example Tables

Departments

DepartmentID	Bonus
1	5000
2	10000

Employees

EmployeeID	DepartmentID	Salary
101	1	45000
102	2	50000

Query

```
UPDATE E
SET Salary = Salary + D.Bonus
FROM Employees E
INNER JOIN Departments D
ON E.DepartmentID = D.DepartmentID;
```

Result

EmployeeID	Salary
101	50000
102	60000

Real-World Uses

- Updating exchange rates
- Applying bonuses
- Synchronizing data

1 1 Common Mistakes

Missing WHERE Clause

Bad:

```
UPDATE Employees
SET Salary = 100000;
```

Updates every row.

Wrong Condition

Bad:

```
WHERE EmployeeID = 1000
```

When ID does not exist.

Result:

```
0 rows affected
```

Updating Wrong Column

Bad:

```
SET EmployeeName = 50000
```

Data type mismatch.

Not Checking Data First

Always run:

```
SELECT *  
FROM Employees  
WHERE EmployeeID = 101;
```

before updating.

1 2 Best Practices

Always Use WHERE

Good:

```
UPDATE Employees  
SET Salary = 60000  
WHERE EmployeeID = 101;
```

Verify Rows First

Run SELECT before UPDATE.

Test in Development Environment

Never test directly in production.

Use Transactions

For critical updates.

Example:

```
BEGIN TRANSACTION;  
  
UPDATE Employees  
SET Salary = Salary + 5000;  
  
ROLLBACK;
```

Review results before COMMIT.

Backup Important Data

Before large updates.

Update Only Necessary Columns

Avoid unnecessary modifications.

Professional Workflow

Step 1

Verify records.

```
SELECT *  
FROM Employees  
WHERE Department='IT';
```

Step 2

Update.

```
UPDATE Employees
SET Salary = Salary + 5000
WHERE Department='IT';
```

Step 3

Verify again.

```
SELECT *
FROM Employees
WHERE Department='IT';
```

1 3 Real-World Examples

Employee Salary Increase

```
UPDATE Employees
SET Salary = Salary * 1.10
WHERE Department = 'IT';
```

Product Price Update

```
UPDATE Products
SET Price = Price * 1.05;
```

Change Customer City

```
UPDATE Customers
SET City = 'Delhi'
```

```
WHERE CustomerID = 101;
```

Activate Users

```
UPDATE Users  
SET IsActive = 1  
WHERE IsActive = 0;
```

Order Status Update

```
UPDATE Orders  
SET OrderStatus = 'Delivered'  
WHERE OrderID = 1001;
```



Summary

In this module, you learned:

- ✓ UPDATE Statement
 - ✓ Single Row Updates
 - ✓ Multiple Row Updates
 - ✓ WHERE Clause with UPDATE
 - ✓ Multiple Column Updates
 - ✓ Expressions in UPDATE
 - ✓ CASE with UPDATE
 - ✓ UPDATE Using JOIN
 - ✓ Common Mistakes
 - ✓ Professional Best Practices
-



Practice Questions

Theory

1. What is UPDATE?
 2. Why is WHERE important in UPDATE?
 3. How do you update multiple columns?
 4. What happens if WHERE is omitted?
 5. How can expressions be used in UPDATE?
 6. What is UPDATE with CASE?
 7. What is UPDATE with JOIN?
 8. Why should SELECT be run before UPDATE?
 9. Why use transactions?
 10. What are common UPDATE mistakes?
-

Practical Exercises

Task 1

Update salary of EmployeeID 101.

Task 2

Increase all salaries by ₹5000.

Task 3

Increase IT employee salaries by 10%.

Task 4

Update:

```
Salary  
City
```

together.

Task 5

Use CASE to provide different salary increases.

Task 6

Use UPDATE with JOIN.

Challenge Project

Create:

Employees
Departments

Apply:

- Department-specific salary hikes
- Bonus updates from Department table
- Multi-column updates
- Transaction-based updates



Next Module

→ Module 14: DELETE Statement

Topics Covered:

- DELETE Syntax
- DELETE with WHERE
- Deleting Single Rows
- Deleting Multiple Rows
- DELETE vs TRUNCATE
- DELETE vs DROP
- Safe Deletion Practices
- Transactions with DELETE